

DOCUMENTAÇÃO TÉCNICA

VulnScan Toolkit

Orquestrador Modular de Varredura de Vulnerabilidades Web

Versão 0.1 (MVP) · Julho de 2026 · CIESC / Projeto de Inovação

Autor: Higor Silva

1. Visão Geral

O VulnScan Toolkit é um orquestrador modular e containerizado que automatiza a varredura de vulnerabilidades em aplicações web, integrando ferramentas de segurança open-source consolidadas (sqlmap, OWASP ZAP, Nikto, Nuclei) sob uma única interface de linha de comando. Ele normaliza a saída de cada ferramenta em um modelo de dados comum e gera um relatório consolidado e profissional (HTML e PDF) contendo sumário executivo, criticidade baseada em CVSS, detalhamento técnico e recomendações de mitigação.

O objetivo não é reinventar motores de varredura de vulnerabilidades, mas eliminar a fricção operacional de rodar múltiplas ferramentas desconexas manualmente, reconciliar formatos de saída inconsistentes e montar um relatório à mão para as partes interessadas.

2. Arquitetura

O sistema é organizado em quatro camadas desacopladas:

Camada	Responsabilidade	Local
CLI / Ponto de entrada	Parsing de argumentos, confirmação de autorização, inicialização da execução	<code>src/main.py</code>
Núcleo (Core)	Controle de escopo, orquestração, modelo normalizado <code>Finding</code>	<code>src/core/</code>
Módulos de Varredura	Adaptadores que invocam ferramentas externas e normalizam sua saída	<code>src/modules/</code>
Motor de Relatórios	Agregação, estatísticas, renderização HTML/PDF	<code>src/report/</code>

O núcleo é propositalmente desacoplado de qualquer ferramenta de varredura específica. Todo módulo implementa o mesmo contrato (`BaseScanModule.run()` -> `list[Finding]`), de forma que o orquestrador e o motor de relatórios nunca precisam saber qual ferramenta subjacente produziu um resultado. Adicionar, remover ou substituir uma ferramenta exige alterar apenas seu módulo e sua entrada no registro.

2.1 Fluxo de dados

1. O usuário invoca a CLI informando URL alvo, perfil de varredura e a flag explícita de autorização.
2. O `ScopeGuard` verifica o alvo contra a allowlist em `config/scope.yaml`; a execução é abortada se o host não estiver autorizado.
3. O `ScanOrchestrator` resolve a lista de módulos (a partir do perfil ou de um override explícito via `--modules`) e executa cada módulo sequencialmente, capturando a saída bruta da ferramenta para fins de auditoria.

4. Cada módulo normaliza seus achados em objetos `Finding` (severidade, vetor/pontuação CVSS, endpoint, parâmetro, evidência, recomendação).
5. O `ReportGenerator` agrega todos os achados, calcula estatísticas de sumário e renderiza o template Jinja2 em HTML, convertendo opcionalmente para PDF via WeasyPrint.

3. Stack Tecnológica

Componente	Escolha	Justificativa
Linguagem	Python 3.11+	Ecosistema maduro de subprocess/parsing; linguagem nativa da maioria das ferramentas ofensivas
Configuração	PyYAML	Arquivos de escopo/parâmetros editáveis por humanos
Templating	Jinja2	Separação clara entre lógica do relatório e apresentação
Renderização PDF	WeasyPrint	Converte o mesmo HTML/CSS para PDF sem depender de binários externos como wkhtmltopdf
Containerização	Docker (debian:bookworm-slim)	Isola todas as dependências das ferramentas ofensivas do host (ex: Kali), evitando conflitos de bibliotecas

4. Módulos de Varredura

Módulo	Classe de vulnerabilidade	Ferramenta subjacente	Integração
<code>recon</code>	Misconfiguração geral / exposição de informação	Nikto	CLI, saída JSON
<code>sqli</code>	SQL Injection	sqlmap	CLI (<code>--batch</code>), parsing de stdout
<code>xss</code>	XSS Refletido / Armazenado / DOM	OWASP ZAP (baseline scan)	CLI (<code>zap-baseline.py -J</code>), parsing de alertas JSON
<code>xxe</code>	XML External Entity	Nuclei (tag <code>xxe</code>)	CLI (<code>-jsonl</code>), parsing de JSON linha a linha

Os perfis de varredura agrupam conjuntos de módulos para casos de uso comuns: **quick** executa `recon` + `xss` para uma passada rápida, **full** executa os quatro módulos, e **custom** permite seleção explícita de módulos via `--modules`.

5. Modelo Normalizado de Achado

Todo módulo retorna uma lista de objetos `Finding` (`src/core/finding.py`), o contrato único entre os módulos de varredura e o motor de relatórios:

```
module, title, description, severity (enum),
cvss_vector, cvss_score, endpoint, parameter,
evidence, recommendation, references[], tool_source
```

A severidade usa um enum de cinco níveis (Crítica, Alta, Média, Baixa, Informativa), mapeado de forma consistente independentemente da taxonomia de severidade própria da ferramenta de origem (ex: níveis de risco do ZAP, tags de severidade do Nuclei).

6. Motor de Relatórios

O motor de relatórios (`src/report/generator.py`) executa três etapas: calcula estatísticas agregadas (contagem de achados por severidade, CVSS médio, módulos executados), ordena os achados por criticidade e renderiza `report/templates/report.html.j2`. O documento resultante contém quatro seções: Sumário Executivo, Visão de Criticidade (distribuição visual por severidade), Detalhamento Técnico (endpoint, parâmetro, evidência e CVSS por achado) e Recomendações de Mitigação. O HTML é sempre gerado; o PDF é gerado sob demanda via WeasyPrint, usando o mesmo HTML/CSS para manter os dois formatos visualmente consistentes.

7. Controles de Segurança

O controle de escopo é obrigatório, não opcional. O ScopeGuard lê uma allowlist de hosts autorizados (`config/scope.yaml` , com suporte a padrões wildcard) e recusa executar qualquer módulo contra um alvo que não esteja listado. Além disso, a CLI exige a flag explícita `--i-have-authorization` antes de qualquer varredura, como um ponto de fricção deliberado contra uso indevido acidental.

Toda a saída bruta das ferramentas (stdout/stderr/JSON) é preservada por execução em `reports/<run_id>/<módulo>/` para fins de rastreabilidade e auditoria.

8. Deploy

A ferramenta é distribuída como uma única imagem Docker construída a partir do `Dockerfile` fornecido, instalando sqlmap e Nikto via apt, Nuclei via `go install` , e o script baseline do OWASP ZAP a partir do pacote oficial de release. O `docker-compose.yml` monta `./reports` e `./config` como volumes para que a saída dos scans e a configuração persistam fora do container.

```
docker compose build
docker compose run --rm vulnscan \
  --target https://app-teste.local \
  --profile full \
  --format both \
  --i-have-authorization
```

9. Limitações Conhecidas (MVP)

- Os módulos executam sequencialmente; ainda não há paralelização (planejada para um sprint futuro).
- A saída do sqlmap é parseada via regex sobre o stdout; uma iteração futura consumirá diretamente a saída estruturada do sqlmap para maior robustez.
- Não há histórico/banco de dados persistente de varreduras; cada execução é um diretório independente em `reports/`.
- Não há suporte a varredura autenticada (fluxos de login) no conjunto atual de módulos.